

Construction du cluster Nebula

Ce document couvre la construction du cluster à partir des VM déjà préparées. La préparation des machines (création, installation, réseau, clonage, swap, Docker, vérifications) suit l'**Annexe D** du cahier des charges : *Nebula_AnnexeD_Machines_virtuelles.pdf*.

1. Préparation

Accès aux machines

Connexion via le routeur, avec le fichier `~/.ssh/config` du PC :

```
Host routeur
    HostName 10.210.0.18
    User root

Host manager
    HostName 10.96.237.11
    User labomatics
    ProxyJump routeur
    IdentityFile ~/.ssh/id_ed25519
```

Les blocs `worker1` et `worker2` suivent le même modèle, avec leurs adresses. Le mot de passe demandé d'abord est celui du routeur.

Réglages Docker propres au projet

Sur les trois VM : `daemon.json` complet (en plus des journaux de l'annexe) :

```
sudo tee /etc/docker/daemon.json <<'EOF'
{
    "log-driver": "local",
    "log-opts": {"max-size": "10m", "max-file": "3"},
    "insecure-registries": ["10.96.237.11:5000"],
    "dns": ["1.1.1.1", "8.8.8.8"]
}
EOF
sudo systemctl restart docker
```

- `insecure-registries` : le registry local n'est pas chiffré.
- `dns` : les conteneurs de construction ne peuvent pas utiliser le résolveur `127.0.0.53` de l'hôte.

Sur le manager uniquement : Traefik v3.3 n'accepte que l'API Docker 1.24, alors que Docker 29 exige 1.40 minimum.

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/override.conf <<'EOF'
[Service]
Environment="DOCKER_MIN_API_VERSION=1.24"
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
docker version --format '{{.Server.MinAPIVersion}}'
```

La dernière commande affiche `1.24`.

Dépôt sur le manager

Copie depuis le PC (la version HTTPS de GitHub demande un jeton, qui n'est pas stocké sur les VM) :

```
scp -r C:\Users\louis\Documents\COURS\M2\Cluster\nebula manager:~/  
cd ~/nebula  
chmod +x scripts/build-push.sh scripts/smoke.sh
```

Les droits d'exécution sont perdus lors du passage Windows → Linux. Ils sont enregistrés dans Git (`git update-index --chmod=+x`, puis commit) pour qu'un clone les conserve.

2. Topologie

Topologie : **1 manager + 2 workers**.

Machine	Rôle Swarm	Adresse	Étiquettes
manager	manager (Leader)	10.96.237.11	aucune
worker1	worker	10.96.237.12	<i>db=true, bus=true</i>
worker2	worker	10.96.237.13	aucune

3. Cluster

3.1 Formation

```
docker swarm init --advertise-addr 10.96.237.11
```

Sur worker1 et worker2, avec le jeton affiché :

```
docker swarm join --token <TOKEN> 10.96.237.11:2377
```

Le jeton n'est pas conservé ici. Pour le réafficher : `docker swarm join-token worker`.

Vérification : `docker node ls` affiche trois nœuds **Ready**, dont un **Leader**.

3.2 Étiquettes de placement

```
docker node update --label-add db=true --label-add bus=true worker1
```

La base et le bus sont sur worker1, pour que leurs données restent sur un nœud identifié.

3.3 Registry

Conteneur autonome, publié sur l'adresse interne du manager :

```
docker run -d --name registry --restart=always -p 10.96.237.11:5000:5000  
-v registry-data:/var/lib/registry registry:2  
curl http://10.96.237.11:5000/v2/
```

La réponse est `{}`, depuis chacun des trois nœuds.

3.4 Secrets et configuration

```
cd ~/nebula  
openssl rand -hex 24 | docker secret create pg_password -  
openssl rand -hex 24 | docker secret create bus_password -  
docker config create init_sql db/init.sql
```

- Les secrets sont en **hexadécimal** : le base64 peut contenir `/` ou `+`, qui cassent l'URL de connexion AMQP.

- Les valeurs ne sont jamais dans le dépôt, et ne se relisent pas après création.
- `init_sql` est exécuté une seule fois, au premier démarrage du volume de la base.

3.5 Réseaux

```
docker network create --driver overlay --internal nebula_internal
```

`nebula_internal` est interne : la base, le cache, le bus et les services n'ont aucun accès vers l'extérieur. `edge_public` n'est pas créé à la main : la stack edge s'en charge.

4. Point d'entrée (edge)

```
docker stack deploy -c swarm/stack.edge.yml edge  
docker stack services edge
```

L'edge fourni est déployé sans modification. `make edge` n'est pas utilisé : sa première ligne crée `edge_public` à la main, ce qui entre en conflit avec la stack.

5. Images

```
cd ~/nebula  
make build REGISTRY=10.96.237.11:5000 TAG=v1
```

Tags de livraison, jamais `latest`. Chaque service répond sa version sur `/health`.

Image	Tag	Digest
nebula-comptes	v1	sha256:5d14f92b053f73ec8306a5fb6599e334b632c57e95083737f96868c5d282d753
nebula-publications	v1	sha256:ad844ac23da8f76572f34942f4961a8d32dfe1719f2a6e730dbc4b04f9ba9c0c
nebula-worker-medias	v1	sha256:9becea04c5801104173859a78be3451b8f0b77d6f14075f1914d03aa85f4879c

Code source : commit `b513cbe`.

6. Stack applicative

```
cd ~/nebula  
make deploy REGISTRY=10.96.237.11:5000 TAG=v1  
docker stack services nebula
```

La procédure complète est dans `docs/procedures/01-deploiement-initial.md`.

7. Problèmes rencontrés

Problème	Cause	Solution
<code>Permission denied (publickey)</code>	Image cloud sans mot de passe SSH	Clé publique dans Cloud-Init
<code>/etc/hosts</code> réécrit au démarrage	Gestion par cloud-init	<code>manage_etc_hosts: false</code> dans un fichier <code>zz-...</code>

Problème	Cause	Solution
<i>npm ci</i> échoue au build	DNS <i>127.0.0.53</i> inaccessible aux conteneurs	<i>dns</i> dans <i>daemon.json</i>
<i>Permission denied</i> sur les scripts	Droits perdus depuis Windows	<i>chmod +x</i> , puis commit des droits
Traefik en <i>404</i> puis <i>502</i>	API Docker 1.24 refusée	<i>DOCKER_MIN_API_VERSION=1.24</i>
Worker en <i>EACCES</i> sur <i>/data</i>	Volume <i>root</i> , processus <i>node</i>	Traces dans <i>/home/node/traces</i>
Services qui redémarrent en boucle	RabbitMQ non démarré (<i>_FILE</i> refusé)	Mot de passe lu dans le shell
Validation de stack refusée	<i>\$ (...)</i> interprété par Docker	<i>\$\$ (...)</i> dans le fichier
<i>make edge</i> en conflit	Réseau créé deux fois	<i>docker stack deploy</i> direct

8. Résultats des tests

Scénario	Résultat	Mesure
1 : cluster	Trois nœuds <i>Ready</i> , un Leader	—
2 : redémarrage complet	Cluster reformé sans intervention, services à 3/3, données présentes	5 min 28 s (07:30:48 → 07:36:16)
3 : déploiement depuis zéro	Reste à réaliser avec la procédure 1	—
4 : exposition	Base, cache, bus fermés ; 80 et 8088 sur les trois nœuds ; 5000 sur le manager	Relevé du 06/10
5 : placement	Base et bus sur worker1	—
6 : montée en charge	5 répliquas, 5 hôtes, 12 réponses chacun sur 60	Relevé du 06/10
7 : mise à jour	<i>v2</i> déployée : 179 réponses <i>200</i> , une <i>504</i> non expliquée	À confirmer par le log Traefik
8 : version défectueuse	Échec <i>image not found</i> constaté ; 240 requêtes sur 240 en <i>200</i> ; retour arrière manuel	Environ 23 s
9 : données	Publication présente après redémarrage ; sauvegarde restaurée et comparée	—
10 : ajout de service	Service <i>whoami</i> en stack séparée, route opérationnelle, puis retiré	Moins de deux minutes